

Function and user define function

If a group of statements is repeatedly required then it is not recommended to write these statements every time separately. We have to define these statements as a single unit and we can call that unit any number of times based on our requirement without rewriting. This unit is nothing but function.

```
x=10
y=20
print("Addition of x & y =",x+y)
print("Addition of x & y =",x-y)
print("Addition of x & y =",x*y)
```

```
x=200
y=100
print("Addition of x & y =",x+y)
print("Addition of x & y =",x-y)
print("Addition of x & y =",x*y)
```

```
x=10
y=5
print("Addition of x & y =",x+y)
print("Addition of x & y =",x-y)
print("Addition of x & y =",x*y)
```

O/P:--

```
Addition of x & y = 30
Addition of x & y = -10
Addition of x & y = 200
Addition of x & y = 300
Addition of x & y = 100
Addition of x & y = 20000
Addition of x & y = 15
Addition of x & y = 5
Addition of x & y = 50
```

Note: Now, repeated code can be bound into single unit that is called function.

The **advantages** of function:

1. Maintaining the code is an easy way.
2. Code re-usability.

A **function in Python** is a **block of reusable code** that performs a specific task. You write it once and can **call (use) it many times** in your program.

Type of function:-

1. In-built function
2. User define function

In-built function:-

1. print()
2. type()
3. id()
4. max()
5. min()
6. len()
7. sum()
8. eval() etc.....

User define function:-

Syntex:

```
def fun_name(parameters...):  
    ''' doc string...'''  
  
    Statment1.....  
    Statment2.....  
    Statment3.....  
  
    return (anything)  
  
# call function  
fun_name(arguments....)
```

***required :-** def, func_name, () , :

optional :- parameters , return , arguments

Type of user define function:-

1. Function with no arguments & no return value

```
def func_name():  
    fun_body.....  
    fun_body.....  
    fun_body.....  
  
result = func_name()
```

2. Function with arguments & no return value

```
def func_name(parameters....):  
    fun_body.....  
    fun_body.....  
    fun_body.....  
  
result = func_name(arguments....)
```

3. Function with no arguments & with return value

```
def func_name():  
    fun_body.....  
    fun_body.....  
    fun_body.....  
    return .....  
  
result = func_name()
```

4. Function with arguments & with return value

```
def func_name(parameters....):  
    fun_body.....  
    fun_body.....  
    fun_body.....  
    return .....  
  
result = func_name(arguments....)
```